



소설 작가 분류 AI 경진대회

OB 1조

김경민, 유재은, 전예원, 편한비

목차

- 전처리
- TF-IDF + XGBoost
- TF-IDF + CatBoost
- Embedding + GRU
- BERT

대회 설명

DAICON

DAKER

커뮤니티

대회

학습

더보기

월간 데이콘 소설 작가 분류 AI 경진대회

알고리즘 | NLP | 분류 | 자연어 | LogLoss

₩ 상금 100만 원+애플워치

🕒 2020.10.29 ~ 2020.12.04 17:59 [+ Google Calendar](#)

👤 1,469명 📅 마감

- 작가의 글을 분석하여 특징 도출하는 문체 분석 알고리즘 개발

Train			Test		
Notes			Notes		
train.csv(14MB) : (54879, 3)			test_x.csv(10MB) : (19617, 2)		

Train			Test		
index	text	author	index	text	
0	He was almost choking. There was so much, so m...	3	0	"Not at all. I think she is one of the most ch...	
1	"Your sister asked for it, I suppose?"	2	1	"No," replied he, with sudden consciousness, "...	
2	She was engaged one day as she walked, in per...	1	2	As the lady had stated her intention of scream...	
3	The captain was in the porch, keeping himself ...	4	3	"And then suddenly in the silence I heard a so...	
4	"Have mercy, gentlemen!" odin flung up his han...	3	4	His conviction remained unchanged. So far as I...	
...	...	...	...	...	...
54874	"Is that you, Mr. Smith?" odin whispered. "I h...	2	19612	At the end of another day or two, odin growing...	
54875	I told my plan to the captain, and between us ...	4	19613	All afternoon we sat together, mostly in silen...	
54876	"Your sincere well-wisher, friend, and sister...	1	19614	odin, having carried his thanks to odin, proc...	
54877	"Then you wanted me to lend you money?"	3	19615	Soon after this, upon odin's leaving the room...	
54878	It certainly had not occurred to me before, bu...	0			

- train 데이터 : index, text, author
 - test 데이터 : index, text
- > author를 예측함

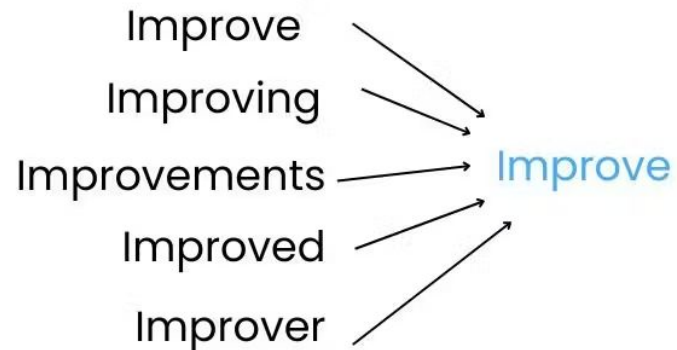
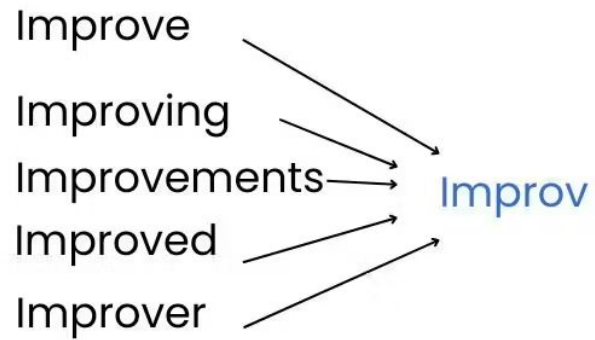


1.전처리

표제어 추출

- 단순히 어미를 잘라내는 어간 추출과 달리, 표제어 추출은 사전적 의미를 유지함
 1. 정확성: studies -> study
 2. 문맥 활용: 품사(POS) 정보를 입력받아 동사와 명사를 구분하여 처리
 3. 사전 조회가 필요하여 어간 추출보다 연산 비용이 높음

Stemming vs Lemmatization



표제어 추출

- NLTK의 WordNetLemmatizer는 기본적으로 모든 단어를 명사로 처리함
 - > Treebank 형식의 POS 태그를 WordNet 형식에 이해할 수 있는 4가지(J,V,N,R)로 변환함

```
def get_wordnet_pos(treebank_tag):
    """NLTK 품사 태그를 WordNet 품사 태그로 변환"""
    if treebank_tag.startswith('J'): return wordnet.ADJ
    elif treebank_tag.startswith('V'): return wordnet.VERB
    elif treebank_tag.startswith('N'): return wordnet.NOUN
    elif treebank_tag.startswith('R'): return wordnet.ADV
    else: return wordnet.NOUN

def apply_lemmatization(text):
    """품사를 고려한 표제어 추출(Lemmatization)"""
    words = text.split()
    pos_tags = pos_tag(words)
    lemmatized_words = [lemmatizer.lemmatize(word, get_wordnet_pos(tag)) for word, tag in pos_tags]
    return " ".join(lemmatized_words)
```

최적 max_length 설정

- 문장당 최대 단어 길이를 242로 설정하면 이상치를 제외한 99% 데이터를 커버함
 - 단순히 최대 길이를 기준으로 모델을 돌리면 소수의 극단적인 이상치 때문에 모델 전체의 메모리 낭비가 심해짐
- > 전체 데이터의 99%를 안전하게 포괄하면서도 연산 효율을 챙김

```
# 문장 길이 분포 확인 및 max_length 설정
train['word_len'] = train['text'].apply(lambda x: len(x.split()))
quantiles = train['word_len'].quantile([0.5, 0.75, 0.9, 0.95, 0.99])
print("📊 학습 데이터 단어 길이(word_len) 분포:\n", quantiles)

# 99% 혹은 95% 데이터를 포괄하는 길이를 max_length로 지정
max_length = int(quantiles[0.99])
print(f"✅ 권장 max_length 지정: {max_length}")
```

```
📊 학습 데이터 단어 길이(word_len) 분포:
0.50    22.0
0.75    50.0
0.90   111.0
0.95   158.1
0.99   242.0
Name: word_len, dtype: float64
✅ 권장 max_length 지정: 242
```

수치형 피처 엔지니어링

1. 기초 통계 피처

- 'len': 문장의 순수 글자 수(공백 포함 글자 길이)
- 'words': 문장 내 단어의 개수

-> 단문 글인지, 장문 글인지 파악

```
def extract_features(df):  
    """텍스트 기반 수치형 피처 추출 (Train/Test 동일 적용을 위해 함수화)"""  
    df_features = pd.DataFrame(index=df.index)  
  
    # 기초 통계  
    df_features['len'] = df['text'].apply(len)  
    df_features['words'] = df['text'].apply(lambda x: len(x.split()))
```

수치형 피처 엔지니어링

2. 비율 관련 피처

- `comma_ratio`: 전체 글자 수 대비 쉼표(,)가 등장하는 비율
-> 높으면, 호흡이 길고 부연 설명이 많은 문체일 확률이 높음
- `word_len_avg`: 단어당 평균 글자 길이 (총 글자 수 / 단어 수)
-> 높으면, 대화체(hi, ok 등의 짧은 단어) 전문적이거나 길이가 긴 고급 어휘(establishment 등)를 많이 썼을 확률이 높음

```
# 비율 관련 피처
```

```
df_features['comma_ratio'] = df_features['comma_cnt'] / (df_features['len'] + 1)  
df_features['word_len_avg'] = df_features['len'] / (df_features['words'] + 1)
```

수치형 피처 엔지니어링

3. 어휘 다양성 및 스톱워드 밀도

- **ttr** (Type-Token Ratio): 전체 단어 수 대비 중복을 제거한 고유 단어 수의 비율
-> 1에 가까울수록 매번 새로운 단어를 사용하는 어휘력이 풍부한 글임
- **stop_density**: 문장 내에 의미 없는 기능어(불용어: **is**, **at**, **which** 등)가 얼마나 자주 등장하는지 비율

```
# 어휘 다양성 및 스톱워드 밀도
df_features['ttr'] = df.apply(lambda x: len(set(x['text'].split())) / (len(x['text'].split()) + 1), axis=1)
df_features['stop_density'] = df['text'].apply(lambda x: len([w for w in x.split() if w.lower() in stop_words]) / (len(x.split()) + 1))
```

수치형 피처 엔지니어링

4. 구두점 및 특수문자 카운팅

```
# 구두점 및 특수문자
df_features['comma_cnt'] = df['text'].apply(lambda x: x.count(','))
df_features['dot_cnt'] = df['text'].apply(lambda x: x.count('.'))
df_features['ques_cnt'] = df['text'].apply(lambda x: x.count('?'))
df_features['excl_cnt'] = df['text'].apply(lambda x: x.count('!'))
df_features['quote_cnt'] = df['text'].apply(lambda x: x.count('"') + x.count("'"))
df_features['special_punc'] = df['text'].apply(lambda x: x.count('?")') + x.count('!"') + x.count(',"'))
```

수치형 피처 엔지니어링

4. 구두점 및 특수문자 카운팅

- `comma_cnt` / `dot_cnt` : 쉼표 및 마침표 개수
 - > 문장의 호흡과 리듬을 측정함. 쉼표 비율이 높으면 수식어나 부연 설명이 많은 만연체 문장이고, 마침표가 많으면 짧고 명확하게 끊어 치는 간결체 문장임
- `ques_cnt` / `excl_cnt` : 물음표 및 느낌표 개수
 - > 글의 분위기와 감정의 격양도를 감지함. 물음표가 많으면 의문문, 대화체, 혹은 독자에게 질문을 던지는 논설문 형태일 수 있음. 느낌표가 많으면 감정이 강하게 이입된 감탄문이거나 강조형 문장임.
- `quote_cnt` : 따옴표 개수
 - > 높다는 것은 인용구가 많거나 대화(Dialogue) 중심의 서사 구조를 가지고 있다는 결정적인 증거가 됨
- `special_punc` : 복합 구두점 패턴
 - > `?", !, .` 같이 구두점과 따옴표가 연달아 나오는 특수한 패턴을 잡아냄. 주로 소설이나 수필 등에서 대사가 끝나는 지점에 사용하고, 일반적인 설명문이나 딱딱한 레포트에서는 거의 전무한 패턴이므로 문체를 분류하는 데 매우 유용함

수치형 피처 엔지니어링

4. 대명사 및 관사 비중

- `pronoun_ratio` : 대명사 비중

-> 주로 인물 간의 관계나 서사가 얽혀 있는 스토리텔링(소설, 수필)에서 압도적으로 높게 나타남

- `article_ratio` : 관사 비중

-> 관사는 뒤에 반드시 '명사'를 동반함. 따라서 관사 비중이 높다는 것은 문장 내에 명사구 중심의 객관적 정보나 사물, 개념에 대한 설명이 밀도 있게 채워져 있음

```
# 대명사 및 관사 비중
pronouns = ['he', 'she', 'it', 'they', 'him', 'her', 'them', 'his', 'hers', 'their']
articles = ['the', 'a', 'an']
df_features['pronoun_ratio'] = df['text'].apply(lambda x: len([w for w in x.split() if w.lower() in pronouns]) / (len(x.split()) + 1))
df_features['article_ratio'] = df['text'].apply(lambda x: len([w for w in x.split() if w.lower() in articles]) / (len(x.split()) + 1))
```

수치형 피처 엔지니어링

5. 품사 카운팅 및 동적 결합

[get_pos_counts(text) 내부 함수]

1) word_tokenize(text)

: 문장을 형태소 단위로 더 정밀하게 쪼갬

2) pos_tag(tokens)

: 각 토큰에 정확한 품사를 붙여줌

3) for word, tag in tags

: 루프를 돌며 동사(VB), 형용사(JJ), 명사(NN) 계열로 시작하는 품사 태그의 개수를 각각 카운트하여 딕셔너리에 저장하고, 이를 Pandas의 Series 형태로 반환함

```
# 품사 카운팅 (VB, JJ, NN)
def get_pos_counts(text):
    tokens = word_tokenize(text)
    tags = pos_tag(tokens)
    counts = {'VB': 0, 'JJ': 0, 'NN': 0}
    for word, tag in tags:
        if tag.startswith('VB'): counts['VB'] += 1
        elif tag.startswith('JJ'): counts['JJ'] += 1
        elif tag.startswith('NN'): counts['NN'] += 1
    return pd.Series(counts)

pos_df = df['text'].apply(get_pos_counts)
df_features = pd.concat([df_features, pos_df], axis=1)

return df_features.fillna(0)
```

return df_features.fillna(0)

: 아주 짧은 문장이라 동사나 형용사가 아예 등장하지 않으면 결측치가 생길 수 있으므로 0을 채워 넣음

수치형 피처 엔지니어링

6. 피처 스케일링

- StandardScaler를 이용해 평균 0, 표준편차 1의 표준정규분포로 스케일링을 진행함
- test 데이터 세트에는 fit 과정을 생략하고 transform만 적용하여 현실적인 평가 환경을 구축하고 데이터 누수 문제를 차단함

```
# 수치형 피처 스케일링
scaler = StandardScaler()
train_num_scaled = scaler.fit_transform(train_num_features)
test_num_scaled = scaler.transform(test_num_features)
```

TF-IDF

TF-IDF 벡터화 사용 이유

- 텍스트 데이터를 머신러닝 모델이 학습할 수 있도록 수치 벡터 형태로 변환
- 단순 등장 빈도가 아니라, 문서 내 **중요도가 높은 단어에 더 큰 가중치**를 부여

TF-IDF

```
# =====  
# 4. TF-IDF 벡터화 및 데이터 병합  
# =====  
# (1) Word TF-IDF (Lemmatization된 텍스트 사용 권장)  
tfidf_word = TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=15000)  
X_train_word = tfidf_word.fit_transform(train['text_lemma'])  
X_test_word = tfidf_word.transform(test['text_lemma'])  
  
# (2) Char TF-IDF (글자 단위 습관 포착, 원본 텍스트 사용)  
tfidf_char = TfidfVectorizer(analyzer='char', ngram_range=(2, 5), min_df=3, max_features=10000)  
X_train_char = tfidf_char.fit_transform(train['text'])  
X_test_char = tfidf_char.transform(test['text'])  
  
# 최종 데이터 셋 (TF-IDF + 수치형 피처 결합)  
X_final = hstack([X_train_word, X_train_char, train_num_scaled])  
X_test_final = hstack([X_test_word, X_test_char, test_num_scaled])  
y = train['author']  
  
print(f"✅ 학습 데이터 최종 피처 개수: {X_final.shape[1]}")  
print(f"✅ 테스트 데이터 최종 피처 개수: {X_test_final.shape[1]}")
```

학습 데이터 최종 피처 개수: 25017
테스트 데이터 최종 피처 개수: 25017

TF-IDF

Word TF-IDF

```
tfidf_word = TfidfVectorizer(ngram_range=(1, 2), min_df=3,  
                             max_features=15000)
```

사용 방식

- Lemmatization(표제어 추출)된 텍스트 사용
- 단어 단위 unigram + bigram 적용

적용 이유

- 단일 단어 뿐 아니라 단어 조합의 의미까지 반영 가능
- 너무 드물게 등장하는 단어는 제거하여 노이즈 감소
- 최대 feature 수 제한으로 과적합 및 메모리 사용 방지

TF-IDF

Char TF-IDF

```
tfidf_char = TfidfVectorizer(analyzer='char', ngram_range=(2, 5), min_df=3,  
                             max_features=10000)
```

사용 방식

- 원본 텍스트 기반 문자 단위 분석
- 2~5글자 조합(character n-gram) 사용

적용 이유

- 작성 스타일이나 철자 패턴 등 세부적인 특징 포착 가능
- 오타, 접미사, 문제 차이에도 강건함
- 저자 식별에 효과적

최종 데이터

최종 데이터 구성

- TF-IDF + 수치형 피처 결합
- 의미 정보 + 문체 정보 + 통계적 특징을 함께 활용
- 서로 다른 특징을 결합하여 모델 성능 향상 기대



2.모델링

CatBoost

선택 이유

- Gradient Boosting 기반의 고성능 머신러닝 모델
- 복잡한 비선형 패턴 학습에 강점
- 텍스트 기반 고차원 sparse 데이터에서도 안정적인 성능

CatBoost

데이터 분할

```
X_train, X_val, y_train, y_val = train_test_split(  
    X_final, y, test_size=0.2, random_state=42, stratify=y  
)
```

CatBoost 주요 설정 + GPU 사용

- Iterations = 3000: 충분한 반복 학습을 통해 성능 최적화
- Learning_rate = 0.03: 학습 속도를 낮춰 더 안정적으로 수렴하도록 설정
- Depth = 6: 트리 복잡도를 적절히 제한하여 과적합 방지
- Early_stopping_rounds = 200: 성능 향상이 없으면 조기 종료,
불필요한 학습 및 과적합 방지
- Task_type = 'GPU': GPU를 활용해 데이터 학습 속도 향상

```
cat_model = CatBoostClassifier(  
    iterations=3000,  
    learning_rate=0.03,  
    depth=6,  
    loss_function='MultiClass',  
    eval_metric='MultiClass',  
    random_seed=42,  
    verbose=100,  
    early_stopping_rounds=200,  
  
    # 코랩 GPU 활성화 핵심 옵션  
    task_type='GPU',  
    devices='0',  
    bootstrap_type='Poisson'  
)
```

CatBoost

모델 학습

```
cat_model.fit(  
    X_train, y_train,  
    eval_set=(X_val, y_val),  
    use_best_model=True  
)
```

- Validation 데이터를 이용하여 성능 모니터링
- 가장 성능이 좋은 시점의 모델 자동 저장

CatBoost

최종 예측 및 제출

```
test_preds = cat_model.predict_proba(X_test_final)
submission = pd.read_csv('sample_submission.csv')
submission.iloc[:, 1:] = test_preds
submission.to_csv('colab_catboost_submission.csv', index=False)
```

✓ 최종 Validation Log Loss: 0.6775

리더보드 점수: 0.5201070822
0.5341076547

CatBoost

추가 성능 개선 실험

- Stratified K-Fold 교차검증
- Fold별 예측 앙상블
- 추가 정규화 및 튜닝: 과적합 방지용 규제 추가, learning_rate를 줄여 더 천천히 학습, iterations를 늘려 더 많은 반복 학습

But, 성능 하락

- 고차원 sparse 데이터에서는 복잡한 모델이 오히려 노이즈까지 학습할 가능성 존재

XGBoost

1. 필요한 라이브러리 호출

```
!pip install xgboost -q
```

```
import numpy as np  
import pandas as pd
```

```
from xgboost import XGBClassifier  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import log_loss
```

XGBoost

2. 데이터 형식 정리

- `X_final`, `X_test_final`은 텍스트 벡터화 과정에서 희소행렬 형태가 되므로 CSR 형식으로 변환함
-> CSR 형식은 대부분의 값이 0인 데이터를 효율적으로 저장할 수 있어 메모리 사용량을 줄여줌
- 타깃 변수 `y`는 정수형으로 변환

```
# sparse matrix 형식 변환
X_final = X_final.tocsr()
X_test_final = X_test_final.tocsr()

print("X_final 타입:", type(X_final))
print("X_test_final 타입:", type(X_test_final))

# y 처리
y_encoded = y.astype(int)
num_classes = y_encoded.nunique()

print("클래스 개수:", num_classes)
print("y 고유값:", sorted(y_encoded.unique()))

X_final 타입: <class 'scipy.sparse._csr.csr_matrix'>
X_test_final 타입: <class 'scipy.sparse._csr.csr_matrix'>
클래스 개수: 5
y 고유값: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4)]
```

XGBoost

3. 데이터 분할

- 전체 학습 데이터를 train 80%, validation 20%로 분할
- `stratify=y_encoded`는 클래스 비율이 한쪽으로 치우치지 않게 유지함

4. XGBoost 모델 설정

- `multi:softprob`: 각 클래스별 확률을 출력하는 다중분류 설정
- `mlogloss`: 다중분류 LogLoss 기준으로 성능을 확인함

5. 모델 학습 및 최종 예측

- 학습 데이터를 이용해 모델을 학습하고, validation 데이터를 함께 넣어서 학습 중 성능을 확인함
- 최종 테스트 데이터에 대해 클래스별 예측 확률을 생성함

```
X_tr, X_val, y_tr, y_val = train_test_split(  
    X_final,  
    y_encoded,  
    test_size=0.2,  
    stratify=y_encoded,  
    random_state=42)
```

```
model = XGBClassifier(  
    objective='multi:softprob',  
    num_class=num_classes,  
    eval_metric='mlogloss',  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=8,  
    subsample=0.8,  
    colsample_bytree=0.4,  
    tree_method='hist',  
    random_state=42,  
    n_jobs=-1)
```

```
model.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=100)  
  
val_pred = model.predict_proba(X_val)  
val_logloss = log_loss(y_val, val_pred)  
print(f"Validation LogLoss: {val_logloss}")  
  
test_pred = model.predict_proba(X_test_final)
```

XGBoost

7. 최종 예측 결과

- 단일 train/validation 방식을 사용한 결과
너무 높은 logloss 값이 나옴
- 추가성능 개선 실험: 5-fold CV 방식
 - > 성능을 크게 향상시키지 못함
 - > 시간이 너무 오래 걸림

	public점수	private점수
xgboost_single_submission.csv	0.8441442359	0.8592530562
메모 추가		
xgboost_5fold_submission.csv	0.8429886113	0.8574272543
메모 추가		

Embedding + GRU

필요한 라이브러리 호출

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Input, Embedding, GRU, Bidirectional, Dense, Dropout, Concatenate
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split
```

- 데이터 전처리 및 배열 연산을 위한 **numpy** 라이브러리 탑재
- 텍스트 토큰화 및 패딩 처리를 위한 **Keras** 전처리 모듈 호출
- 딥러닝 아키텍처 설계를 위한 **Functional API** (**Model**, **Input**, **Embedding**, **GRU**, **Concatenate** 등) 구성
- 과적합 방지를 위한 **EarlyStopping** 및 데이터 분할용 **train_test_split** 활용

Embedding + GRU

텍스트 시퀀스 데이터 준비 (Tokenization & Padding)

```
# =====  
# 1. 시퀀스 데이터 준비 (Tokenization & Padding)  
# =====  
vocab_size = 20000 # 사용할 최대 단어 개수  
max_len = 242      # 이전 단계에서 구한 max_length (99% 커버)  
  
# 딥러닝은 원형(Lemma)보다 원본 텍스트(시제, 복수형 포함)를 쓰는 것이 문맥 파악에 유리할 때가 많습니다.  
# 여기서는 train['text']를 기준으로 진행합니다.  
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")  
tokenizer.fit_on_texts(train['text'])  
  
# 텍스트를 숫자 시퀀스로 변환  
train_sequences = tokenizer.texts_to_sequences(train['text'])  
test_sequences = tokenizer.texts_to_sequences(test['text'])  
  
# 길이를 max_len으로 맞춤 (짧으면 뒤에 0을 채우고, 길면 뒤를 자름)  
X_train_seq = pad_sequences(train_sequences, maxlen=max_len, padding='post', truncating='post')  
X_test_seq = pad_sequences(test_sequences, maxlen=max_len, padding='post', truncating='post')
```

- 하이퍼파라미터 설정: `vocab_size = 20000` (최대 단어 개수), `max_len = 242` (전체 데이터의 99% 커버)
- 원형(Lemma) 대신 원본 텍스트 사용: 시제나 복수형 형태가 작가의 문맥적 특성을 파악하는 데 더 유리하다고 판단
- 정수 인코딩: `texts_to_sequences`를 통해 자연어를 숫자 벡터 시퀀스로 변환
- 일관된 패딩 처리: 길이를 동일하게 맞추기 위해 뒷부분에 0을 채우는 `padding='post'` 기법 적용

Embedding + GRU

데이터셋 분할 및 다중 피쳐 결합 준비

```
# =====  
# 2. 검증 셋 분리 (시퀀스 데이터와 수치형 데이터 함께 묶기)  
# =====  
y = train['author']  
# 이전 단계에서 만든 스케일링된 수치형 피쳐: train_num_scaled, test_num_scaled  
  
X_seq_train, X_seq_val, X_num_train, X_num_val, y_train, y_val = train_test_split(  
    X_train_seq, train_num_scaled, y, test_size=0.2, random_state=42, stratify=y  
)
```

- Target 데이터 설정: `y = train['author']` (5명의 작가 클래스)
- 수치형 피쳐 준비: 이전 단계에서 스케일링이 완료된 수치 데이터 활용 (`train_num_scaled`)
- 다중 입력 피쳐 동시 분할: 텍스트 시퀀스(`X_train_seq`)와 수치형 피쳐(`train_num_scaled`)를 동일한 인덱스로 `split`
- `stratify=y` 설정을 통해 학습셋과 검증셋의 작가별 클래스 비율을 균일하게 유지

Embedding + GRU

```
# 3. 하이브리드 GRU 모델 구축 (Functional API)
# =====
# --- [가지 1] 텍스트 시퀀스 처리 (Embedding -> Bi-GRU) ---
text_input = Input(shape=(max_len,), name='text_input')
# 임베딩 차원(output_dim)은 보통 128, 256을 사용합니다.
embedding_layer = Embedding(input_dim=vocab_size, output_dim=128)(text_input)
# 양방향(Bidirectional) GRU를 사용하여 앞뒤 문맥을 모두 파악합니다.
gru_layer = Bidirectional(GRU(64, return_sequences=False))(embedding_layer)
text_dropout = Dropout(0.5)(gru_layer)

# --- [가지 2] 수치형 피쳐 처리 ---
num_features_dim = train_num_scaled.shape[1]
num_input = Input(shape=(num_features_dim,), name='num_input')
num_dense = Dense(32, activation='relu')(num_input)

# --- [병합] GRU가 파악한 문맥 + 수치형 통계 결합 ---
concat_layer = Concatenate()([text_dropout, num_dense])

# --- [최종 출력층] ---
dense_1 = Dense(64, activation='relu')(concat_layer)
dropout_1 = Dropout(0.3)(dense_1)
# 작가가 5명이므로 softmax 사용
output = Dense(5, activation='softmax', name='output')(dropout_1)

# 모델 컴파일
model_gru = Model(inputs=[text_input, num_input], outputs=output)
model_gru.compile(
    loss='sparse_categorical_crossentropy', # y가 One-hot이 아닌 정수형(0~4)일 때 사용
    optimizer='adam',
    metrics=['accuracy']
)
```

모델 아키텍처 파이프라인 :

- **[Branch 1] 텍스트 처리:** Input(242) → Embedding(128) → Bidirectional GRU(64) → Dropout(0.5)
- **[Branch 2] 수치형 처리:** Input → Dense(32, ReLU)
- **[병합 및 출력]:** Concatenate → Dense(64, ReLU) → Dropout(0.3) → Dense(5, Softmax)

컴파일 설정: Loss =

`sparse_categorical_crossentropy` (정수형

레이블 대응), Optimizer = `adam`

Embedding + GRU

```
# =====
# 4. 모델 학습 (Training)
# =====
# 검증 손실(val_loss)이 3번 연속 안 떨어지면 멈추고, 가장 좋았던 가중치로 복구
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

print("🚀 GRU 딥러닝 모델 학습 시작...")
history = model_gru.fit(
    {'text_input': X_seq_train, 'num_input': X_num_train}, y_train,
    validation_data=({'text_input': X_seq_val, 'num_input': X_num_val}, y_val),
    epochs=15,
    batch_size=64,
    callbacks=[early_stop]
)

# =====
# 5. 예측 및 결과 저장
# =====
# Test 데이터 예측 (확률값 반환)
gru_test_probs = model_gru.predict({'text_input': X_test_seq, 'num_input': test_num_scaled})

# 제출 파일 생성
submission = pd.read_csv('sample_submission.csv', index_col='index')
submission.iloc[:, :] = gru_test_probs
submission.to_csv('submission_gru_hybrid.csv')
print("🎉 GRU 모델 예측 완료! 'submission_gru_hybrid.csv' 파일이 생성되었습니다.")
```

학습 파라미터 및 제어:

- `epochs=15, batch_size=64`
- `EarlyStopping`: 검증 손실(`val_loss`)이 3회 연속 개선되지 않으면 조기 종료 및 최적 가중치 복원 (`restore_best_weights=True`)

public점수
private점수
0.4147085191
0.4293771643

BERT

- BERT (Bidirectional Encoder Representations from Transformers)

: 구글이 개발한 양방향 문맥 이해 기반 자연어 처리(NLP) 모델로, 텍스트의 의미를 더 정밀하게 파악할 수 있도록 설계된 딥러닝 언어 모델

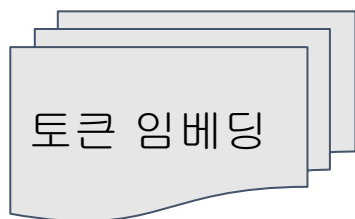
- Transformer 구조 비교 - 번역 예시

Decoder	입력 : 번역할 전체 정보, 지금까지 번역한 정보 → 예측 : 다음으로 번역되어야 하는 단어 [순차적 처리, autoregressive]
Encoder	입력 : 번역할 문장 전체 → 문장 속 단어들 간의 관계 고려 → 결과 : 데이터 재구성, 가중치 반영 [단방향, 양방향 처리]

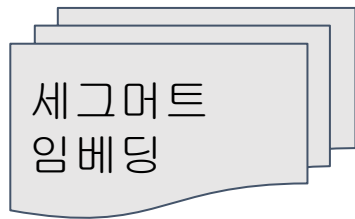
- 양방향 학습으로 문맥을 추출하는 것이 NLP 문제에 유리함 -> Encoder 방식의 BERT
- 선정 이유 : 양방향 데이터 처리 방식을 사용하여 문맥 이해력이 높음 -> 모든 단어가 양쪽의 문맥 동시 참조
- 활용 분야 : 텍스트 분류, 질문 응답, 문장 유사도 측정, 감성 분석, 명명된 엔터티 인식 등 다양한 NLP

BERT 학습 방법

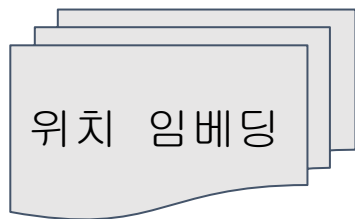
- 구성 : Pretraining + Fine Tuning
 - 임베딩



사람이 사용하는 단어를 딥러닝 모델이 이해할 수 있는 고차원 벡터로 변환



입력 받은 문장의 순서를 모델에게 알려줌



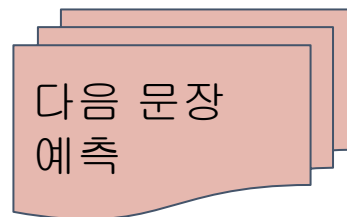
입력된 데이터에서의 순서, 단어가 문장 속에서 위치한 자리를 모델에게 알려줌



- Pretraining



입력 문장의 일부 단어를 지우고 모델이 지워진 단어를 예측하도록 학습
→ 문맥 고려



문장 2개 입력 받아서 첫번째 문장 다음으로 두번째 문장이 오는게 맞는지 이진 분류 학습

BERT 모델 선택

- 최적 모델 선정 - Logloss 비교

- bert-base-uncased: 0.40324
- roberta-base: 0.43890
- albert-base-v2: 0.49053

=> bert-base-uncased 선택 : 모두 소문자로 취급하여 학습하는 모델

```
# 1. 데이터 분할 (양상불 균증을 위해 8:2 유지, 불균형 고려 stratify=y)
train_df, val_df = train_test_split(train, test_size=0.2, random_state=42, stratify=train['author'])

train_dataset = Dataset.from_pandas(train_df[['text_lemma', 'author']].fillna(""))
val_dataset = Dataset.from_pandas(val_df[['text_lemma', 'author']].fillna(""))
test_dataset = Dataset.from_pandas(test[['text_lemma']].fillna(""))
```

1. 작가별 비율을 맞추며 데이터 분할
2. 'text_lemma' 표제어 추출이 완료된 텍스트를 모델에 입력 (그 외 수치화 피쳐는 불필요)
3. 판다스 데이터프레임 -> 허깅페이스 라이브러리 전용 Dataset 객체로 변환

```
# # 2. 모델 및 토크나이저 로드
MODEL_NAME = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model_bert = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=5)
```

↳ BERT 모델 호출

→ 토크나이저 : 단어를 모델이 인식할 수 있는 숫자로 바꿔줌

BERT 모델 학습

```
def tokenize_func(examples):  
    return tokenizer(examples["text_lemma"], padding="max_length", truncation=True, max_length=256)  
  
tokenized_train = train_dataset.map(tokenize_func, batched=True).rename_column("author", "labels")  
tokenized_val = val_dataset.map(tokenize_func, batched=True).rename_column("author", "labels")  
tokenized_test = test_dataset.map(tokenize_func, batched=True)
```

- 딥러닝 토큰화 시 데이터 크기가 모두 동일해야함 → 기준보다 짧은 문장은 0으로 채우는 패딩 실행, 긴 문장은 잘라냄
- 전체 데이터에 규칙 함수 적용

3. 학습 설정 및 실행

```
training_args = TrainingArguments(  
    output_dir="./results_bert_uncased",  
    eval_strategy="epoch",  
    learning_rate=2e-5,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=32,  
    num_train_epochs=3,  
    weight_decay=0.01,  
    fp16=True,  
    load_best_model_at_end=True,  
    metric_for_best_model="log_loss",  
    greater_is_better=False,  
    save_strategy="epoch",  
)
```

```
trainer = Trainer(  
    model=model_bert,  
    args=training_args,  
    train_dataset=tokenized_train,  
    eval_dataset=tokenized_val,  
    compute_metrics=compute_metrics,  
)  
  
trainer.train()
```

학습 시 사용할 허깅페이스의 Trainer
모듈이 labels라는 이름의 컬럼만
인식하므로 변경

Trainer 모듈 : 모델, 데이터, 설정값 입력 시
자동으로 딥러닝 모델 학습시킴

최종 리더보드 결과 - 대략 22위 점수

- Logloss : 0.40324
- public : 0.2115997305
- private: 0.2125124197

BERT - 추가 개선

시도한 개선 방안

1. 로지스틱 회귀 + BERT 앙상블 -> Logloss 증가
2. bert-large-uncased 모델 + 3Fold 교차검증 -> 랩타임 부족, GPU 용량 부족으로 성능 확인 못함

→ 딥러닝 모델 특성상 학습에 많은 시간이 필요하다는 한계에 부딪힘

BERT 정리

1. BERT 모델은 단순 단어 예측이 아닌 맥락을 분석한다. 그러므로 텍스트를 통해 해당 작가를 예측하는 분석에 높은 성능을 보인다.
2. 기존 NLP모델에 비해 많은 파라미터를 갖기 때문에 많은 메모리와 계산이 필요하다는 한계가 존재한다.
3. 모델 내부적으로 데이터를 수치화하여 분석하므로 전처리 시 TF-IDF 와 같이 직접 수치화 피처를 생성할 필요가 없다.